

Projektowanie Oprogramowania Systemów – Projekt

Prowadzący – dr inż. Arkadiusz Szewczyk (p.446 EA)

Założenia projektowe:

Celem projektu jest praktyczne przećwiczenie wykorzystania metod i narzędzi wspomagających tworzenie oprogramowania przedstawionych w trakcie wykładu z przedmiotu.

Proponowane tematy do realizacji:

1. **Automatyczna stacja pogodowa** – oprogramowanie systemu (komputer, czujniki) do monitorowania, gromadzenia i analizy danych pogodowych (temperatura, ciśnienie, siła i kierunek wiatru, opady). Zebrane dane mają być przechowywane w bazie danych i udostępniane w interfejsie graficznym oraz na stronie WWW.
2. **Sterowanie mikroklimatem szklarni** – oprogramowanie systemu (komputer, czujniki, układy wykonawcze) do monitorowania i kontroli parametrów środowiska (temperatura, wilgotność, skład atmosfery (CO₂, N₂) w szklarni. System ma automatycznie utrzymywać zadane wartości poszczególnych parametrów. Zebrane dane mają być przechowywane w bazie danych. Na ich podstawie mają być generowane raporty dostępne w interfejsie graficznym i na stronie WWW.
3. **Monitorowanie parametrów pojazdu** – oprogramowanie systemu (komputer, czujniki, interfejs OBD) do monitorowania, gromadzenia i analizy danych opisujących pracę podzespołów samochodu (temperatura, obroty, ciśnienie wtrysku, ładowanie akumulatora, itp.). Zebrane dane mają być przechowywane w bazie danych i udostępniane w interfejsie graficznym.
4. **Diagnostyka systemów audio** – oprogramowanie systemu (komputer, czujniki, układy pobudzające) do charakteryzowania i diagnostyki systemów audio (źródło dźwięku -> wzmacniacz -> głośniki) pod względem elektrycznym i akustycznym (pasmo przenoszenia i charakterystyki elektroniki i głośników, przesłuchy, zakłócenia, zniekształcenia, szумы). Zebrane dane mają być przechowywane w bazie danych. Na ich podstawie mają być generowane raporty dostępne w interfejsie graficznym.
5. **Tester mikroprocesorów** – oprogramowanie systemu (komputer, czujniki, generatory) do automatycznego testowania mikroprocesorów. Test obejmuje zaprogramowanie mikroprocesora programem testowym, monitorowanie stanów portów wyjściowych, zadawanie stanów na portach wejściowych, sprawdzenie działania układów mikroprocesora (ADC, DAC, przerwania, magistrale komunikacyjne, itp.) Zebrane dane mają być przechowywane w bazie danych. Na ich podstawie mają być generowane raporty dostępne w interfejsie graficznym.
6. **Propozycja własna**

UWAGA!

Proponowane tematy są pretekstem do wykazania się umiejętnościami i wiedzą zdobytymi w trakcie wykładu. Zrealizowanie zadania „do końca” nie jest wymagane do zaliczenia projektu. Wystarczające jest wykonanie fragmentów (określonych modułów) zadania, niezbędnych do wykazania się zdobytymi umiejętnościami. W trakcie realizacji zadania należy:

1. Zidentyfikować klienta, użytkowników końcowych i inne zainteresowane strony tematu (*stakeholders*); zgromadzić i przeanalizować szczegółowe wymagania funkcjonalne i pozafunkcjonalne zadań; określić scenariusze użycia i stworzyć nadającą się do czytania

specyfikację funkcjonalną; zawrzeć w niej diagramy UML (komponentów, klas, aktywności, inne); przedstawić specyfikację do zaliczenia.

2. Uzyskać dostęp do repozytorium systemu kontroli wersji i śledzenia błędów – np. zewnętrzny hosting (GitHub, Bitbucket...); wyznaczyć osobę odpowiedzialną za utrzymanie porządku w repozytorium (*maintainer*); zaplanować strukturę repozytorium mając na uwadze używane środowisko IDE, język programowania, dodatkowe artefakty (grafiki UI, specyfikacje itp.).

3. Stworzyć model UML kluczowych elementów projektowanego rozwiązania; przeanalizować możliwość użycia dodatkowych bibliotek i narzędzi do realizacji zadania; opracować krótką **specyfikację techniczną** podsumowującą te informacje wraz z **harmonogramem i podziałem pracy** w zespole; przedstawić ją do zaliczenia.

4. Podczas implementacji zaangażować odpowiedni nakład pracy na dokumentowanie API (komentarze zgodne z Doxygen) oraz inne komentarze w kodzie, wyjaśniające szczególnie zagmatwane algorytmy, mechanizmy, zależności. Generować dokumentację HTML z komentarzy Doxygen na bieżąco; przedstawić do zaliczenia.

5. Wykryte podczas cykli testowania błędy raportować do systemu śledzenia błędów; opcjonalnie zastosować metodę *test-driven development* do zapewnienia jakości; zademonstrować użycie testów jednostkowych do walidacji kluczowych elementów oprogramowania.

6. Użyć profilera do wykrycia *hot spotów* w opracowanym programie i przeprowadzić próbę ich optymalizacji.

Wykonanie zadania powinno być udokumentowane w postaci sprawozdania zawierającego opis wykonanych operacji, raporty, zrzuty ekranu, i inne elementy dokumentujące realizację poszczególnych punktów.